# MVC・MVP設計

おーたわ

# MVC・MVP設計ってなによ

M:Model
V:View
C:Controller

M:Model
V:View
P:Presenter

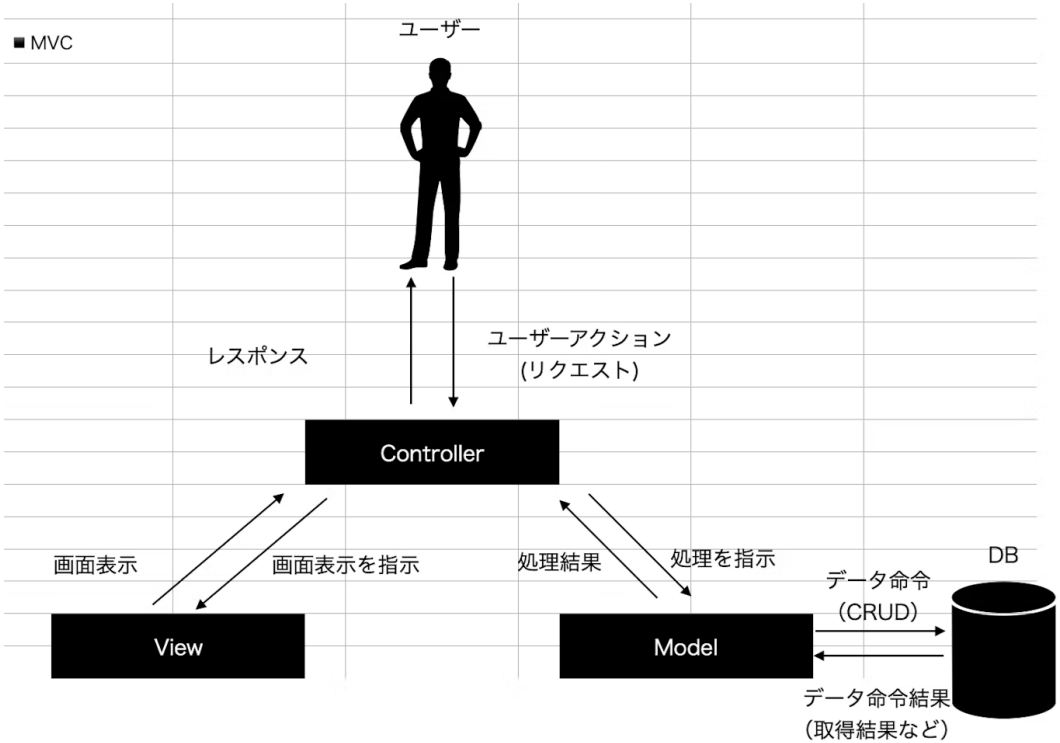
プログラムを役割ごとにに分ける設計手法

# MVC

・**ControllerがViewとModelを制御**
・ViewはModelを参照することがある

・設計が単純

・Controllerが肥大化しがち




画像引用:Qiita:デザインパターン - その2 (MVP, MVVM, MVC) by@H_Stephen

# 実際のコード例

```
public class Model
{
    public int Count { get; private set; }
    public void Add()
    {
        Count++;
    }
}
```

```
public class View : MonoBehaviour
{
    [SerializeField] TMP_Text countText;

    public void Display(int count)
    {
        countText.text = count.ToString();
    }
}
```

```
public class Controller : MonoBehaviour
{
    [SerializeField] View view;
    Model model = new CounterModel();

    void Start()
    {
        view.Display(model.Count);
    }

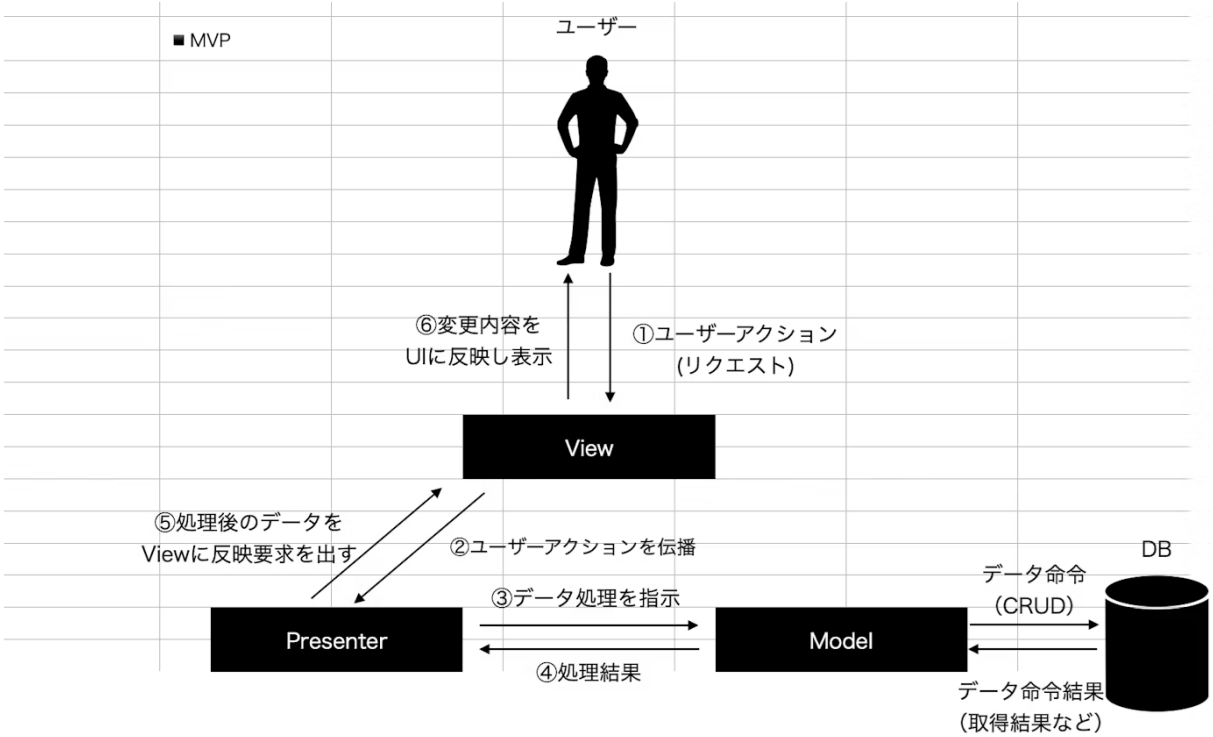
    public void OnAddButton()
    {
        model.Add();
        view.Display(model.Count);
    }
}
```

# MVP

**・ViewとModelの仲介をPresenterが行う**

・ModelとViewが分離しているためテストが容易

・実装コストが高い
・Presenterに処理が集中しやすい




画像引用:Qiita:デザインパターン - その2 (MVP, MVVM, MVC) by@H_Stephen

# 実際のコード例

```
public class CounterModel
{
    public int Count { get; private set; }

    public void Add()
    {
        Count++;
    }
}
```

```
public class CounterView : MonoBehaviour
{
    [SerializeField] TMP_Text countText;
    [SerializeField] Button addButton;
    CounterPresenter presenter;

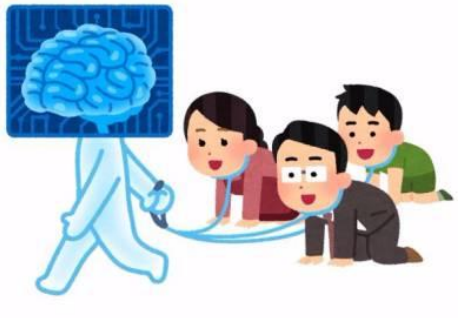
    void Start()
    {
        presenter = new CounterPresenter(this);
        addButton.onClick.AddListener(presenter.OnAddButton);
        presenter.UpdateView();
    }
    public void Display(int count)
    {
        countText.text = count.ToString();
    }
}
```

```
public class CounterPresenter
{
    CounterView view;
    CounterModel model = new CounterModel();
    public CounterPresenter(CounterView view)
    {
        this.view = view;
    }
    public void OnAddButton()
    {
        model.Add();
        UpdateView();
    }
    public void UpdateView()
    {
        view.Display(model.Count);
    }

}
```

開発はチームでやるもの





言われたことをできるように
ならなければいけない

全体像を把握するのが大事

ご清聴ありがとうございました